WEB SEC ANALYZER

A PROJECT REPORT

Submitted by

MUTHUBABU S (311421104051)

JANARTHANAN E (311421104305)

PRASANNA VELAN V (311421104309)

VIGNESH G (311421104314)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING MEENAKSHI COLLEGE OF ENGINEERING, WEST K.K NAGAR

ANNA UNIVERSITY: CHENNAI 600 025

MAY 2025

WEB SEC ANALYZER

A PROJECT REPORT

Submitted by

MUTHUBABU S (311421104051)

JANARTHANAN E (311421104305)

PRASANNA VELAN V (311421104309)

VIGNESH G (311421104314)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING MEENAKSHI COLLEGE OF ENGINEERING, WEST K.K NAGAR

ANNA UNIVERSITY: CHENNAI 600 025

MAY 2025

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report **"WEB SEC ANALYZER"** is the Bonafide work of **"MUTHUBABU S (311421104051), JANARTHANAN E (311421104305), PRASANNA VELAN V (311421104309), VIGNESH G (311421104314)"** who carried out the project work under my supervision**.**

SIGNATURE

DR.CH.SARADA DEVI, M.Tech., Ph.D.,

HEAD OF THE DEPARTMENT, Associate Professor,

Computer Science and Engineering,

Meenakshi College Of Engineering,

West K.K Nagar, Chennai –600078.

SIGNATURE

Mr.S.ANANTHAKOOTHAN, M.E.,

SUPERVISOR, Assistant Professor,

Computer Science And Engineering,

Meenakshi College Of Engineering,

West K.K Nagar , Chennai –600078.

Submitted for the Anna University Project Viva-Voce Examination held on_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

MEENAKSHI COLLEGE OF ENGINEERING

WEST K.K. NAGAR, CHENNAI-600078

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We affirm that the project report titled "WEB SEC ANALYZER" being submitted in partial fulfillment of the requirement for the award of Bachelor of Engineering is the original work carried out by us. It has not formed part of any other project report or dissertation based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

Date: **(Signature of the candidates)**

MUTHUBABU S (311421104051)

JANARTHANAN E (311421104305)

PRASANNA VELAN V (311421104309)

VIGNESH G (311421104314)

I certify that the declaration made by the above candidates is true to the best of my knowledge.

Date: **Name and signature of the Supervisor**

ACKNOWLEDGMENT

First and foremost, we bow our heads to the Almighty for being our guiding light and for his gracious blessings throughout the course of this project.

We express our heartfelt gratitude to our **Chairman**, **Dr. G. Ra. GOKUL, M.B.B.S., M.D.,** Meenakshi College of Engineering, for his dedicated interest in our education and for providing exceptional facilities that support and enrich our learning journey.

We take immense pleasure in expressing our heartfelt thanks to our **Executive Director Dr. N. RENGERAJAN, B.Sc., B.Tech., M.E., Ph.D.** for his constant encouragement and cooperation.

We are deeply grateful to our **Principal, Dr. S. ANANDAKUMAR, M.E., Ph.D., FIE, C.Eng.,** of Meenakshi College of Engineering, for his continuous guidance, encouragement, and support in all our academic endeavors.

We would also like to express our sincere gratitude to the **Head of the Department**, **Dr.CH.SARADA DEVI, M.Tech., Ph.D**, Department of **Computer Science and Engineering**, for her valuable support and motivation throughout the duration of our project.

A special note of thanks goes to our **Supervisor, Mr.S ANANTHAKOOTHAN, M.E**., Assistant Professors, whose expert guidance and encouragement played a crucial role in helping us stay focused and successfully complete this work.

Finally, we extend our thanks to our **family members** for their unwavering support and motivation, as well as to everyone who contributed, directly or indirectly, to the successful completion of this project.

ABSTRACT

The WebSec Analyzer is a cybersecurity solution aimed at detecting phishing websites and analyzing vulnerabilities in web applications. By combining machine learning algorithms and heuristic methods with comprehensive vulnerability scanning, the system identifies and flags malicious domains and security weaknesses. It inspects various parameters including metadata, HTTP headers, SSL configuration, and open ports, while also leveraging AI-based models to detect phishing through URL and content analysis. This dual-layered system enhances online safety and assists users in recognizing potential threats, promoting secure web experiences. WebSec Analyzer not only enhances the detection of phishing attacks but also proactively identifies vulnerabilities that could be exploited by attackers. The system is designed to be accessible, automated, and user-friendly, making it a valuable tool for cybersecurity professionals, developers, and general users alike. Ultimately, this solution promotes safer web interactions and fosters greater awareness of cybersecurity best practices.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
NLP	Natural Language Processing
ML	Machine Learning
API	Application Programming Interface
JSON	JavaScript Object Notation
JWT	JSON Web Token

INTRODUCTION

1.1 GENERAL

With the rapid growth of internet usage, cybersecurity threats like phishing attacks and website vulnerabilities have become more common. Many users and organizations are affected due to a lack of reliable detection tools. Phishing websites trick users into sharing sensitive information, while insecure websites can expose data to attackers.To address these issues, WebSec Analyzer is developed as an all-in-one tool for identifying phishing websites and analyzing web vulnerabilities. It uses machine learning and security auditing methods to help users check the safety and reliability of websites quickly and effectively.

1.2 OBJECTIVE

The main objective of the WebSec Analyzer project is to develop a comprehensive cybersecurity tool that can detect phishing websites and analyze the security vulnerabilities of web applications. The system uses artificial intelligence and heuristic techniques to identify suspicious websites by examining URL patterns, domain information, and website content. In addition, it evaluates the overall security configuration of web applications by scanning for common issues such as missing HTTP security headers, weak SSL implementations, open ports, and other misconfigurations. The aim is to provide users with clear, easy-to-understand insights into the risk level of any website they check. By combining phishing detection and vulnerability analysis into a single platform with a user-friendly interface, WebSec Analyzer empowers users to stay informed and protected in today’s digital environment.

1.3 SUMMARY

The WebSec Analyzer project brings together two main modules — the Phishing Web Detector and the Vulnerability Analyzer — to offer a complete solution for identifying online threats. The Phishing Detector uses artificial intelligence to examine website URLs, content structure, and other key features to determine whether a website is legitimate or a phishing attempt. On the other hand, the Vulnerability Analyzer checks a website's technical details such as HTTP headers, SSL certificate, open ports, and server configurations to find any potential weaknesses. By combining these two powerful tools, the system helps users check if a website is safe to use and whether it has any hidden risks. The project is designed to be easy to use for both technical users and everyday internet users, providing clear and useful results that can help prevent cyberattacks and data breaches.

CHAPTER 2

LITERATURE SURVEY

2.1 GENERAL

A literature survey is an essential part of any research project, as it helps in understanding the existing technologies, methods, and tools developed by other researchers in the same domain. In the case of phishing detection and vulnerability scanning, several techniques have been explored over the years, including blacklist-based detection, rule-based systems, machine learning models, and signature-based vulnerability scanners. The literature survey helps in identifying the strengths and weaknesses of these approaches, and highlights the common challenges such as limited accuracy, lack of real-time analysis, and poor handling of new or unknown threats. By studying these existing systems, we can understand the research gap and build a more efficient and intelligent solution. This section provides a foundation for why a combined tool like WebSec Analyzer, which integrates both phishing detection and vulnerability assessment using AI and heuristic methods, is needed and how it improves upon previous work.

2.2 LITERATURE REVIEW

2.2.1 Title: DEPHIDES: Deep Learning Based Phishing Detection System

Authors: Ozgur Koray Sahingoz, Z. Bubekir Buber, and E. Minkugu

Journal/Year: IEEE Access, January 2024

This paper proposes a deep learning-based phishing detection system named DEPHIDES. The authors present a framework that utilizes advanced classification algorithms to detect phishing attacks effectively. By employing deep learning methods, the system learns patterns from large datasets of phishing and legitimate websites. The study demonstrates the potential of AI in enhancing cybersecurity through intelligent threat classification. While the theoretical framework is well-

developed, the authors did not provide extensive empirical testing, which limits the practical validation of their model.

2.2.2 Title: Enhancing Cybersecurity: A Review and Comparative Analysis of Convolutional Neural Network Approaches for Detecting URL-Based Phishing Attacks

Author: Manika Nanda et al.

Journal/Year: e-Prime - Advances in Electrical Engineering, Electronics and Energy, 2024

This work reviews various deep learning methods, with a specific focus on CNN-based approaches to phishing detection. The authors introduce a novel model named BiLSTM-GHA-CNN, which combines Bidirectional Long Short-Term Memory (BiLSTM) with Convolutional Neural Networks. This hybrid approach significantly enhances feature extraction from phishing URLs and improves detection accuracy. The research highlights the strength of combining temporal and spatial features. However, the model's high computational requirement for training can be a barrier for real-time deployment or resource-constrained environments. make these models more efficient for practical applications in healthcare.

2.2.3 Title:Enhancing Online Security: Machine Learning Methods for Phishing Detection — Literature Survey

Authors: B. Bavithra et al.

Journal/Year: Journal of Web Engineering & Technology, 2023

This literature survey explores various machine learning approaches applied to phishing detection, including Support Vector Machines (SVM) and Deep Learning techniques. The paper reviews how ML methods have adapted to tackle the evolving nature of phishing attacks, emphasizing the flexibility and robustness of algorithms in different scenarios. While the review offers a comprehensive comparison of methods, it lacks the implementation of new techniques or presentation of experimental results. Nevertheless, it serves as a valuable resource for understanding the existing landscape of phishing detection systems.

2.2.4 Title:An Enhanced Deep Learning-Based Phishing Detection Mechanism to Effectively Identify Malicious URLs Using Variational Autoencoders

Authors: Prabakaran et al.

Journal/Year: IET Information Security, 2023

In this research, the authors introduce a deep learning framework that integrates Variational Autoencoders (VAE) with Deep Neural Networks (DNN) for phishing URL detection. The VAE is used for unsupervised feature extraction, while the DNN handles the classification task. This model reduces the need for manual feature engineering and achieves high detection accuracy. The method shows promise for automating phishing detection tasks and improving efficiency.

2.3 SUMMARY

These literature sources collectively provide a solid foundation for understanding the role of machine learning and deep learning in phishing detection. The research supports the feasibility of integrating intelligent models into cybersecurity tools. Despite their benefits, current models face challenges such as high computational demands and lack of empirical validation. These findings have influenced the design and development of the WebSec Analyzer, aiming to overcome these challenges with an optimized and validated system.

CHAPTER 3

SYSTEM ANALYSIS

3.1 GENERAL

This section focuses on analyzing and comparing existing web security tools with the proposed WebSec Analyzer. It highlights the key limitations in current systems, such as lack of integration, outdated detection methods, and limited scanning capabilities. Our system addresses these issues by combining vulnerability analysis and AI-based phishing detection into a single, more efficient platform. This comparison helps in understanding how WebSec Analyzer provides a better and smarter approach to web security.

3.2 EXISTING SYSTEM

Traditional phishing detection and vulnerability scanning tools typically work separately and rely on outdated methods such as blacklists or signature-based detection. These approaches are limited, as they cannot effectively detect new or evolving threats. Most scanners focus only on surface-level checks like open ports or outdated software, often ignoring critical aspects like HTTP headers, SSL certificates, or domain legitimacy. Furthermore, the lack of integration between phishing detection and vulnerability analysis results in incomplete threat assessments, leaving systems exposed to advanced cyberattacks.

3.2.1 EXISTING SYSTEM ARCHITECTURE

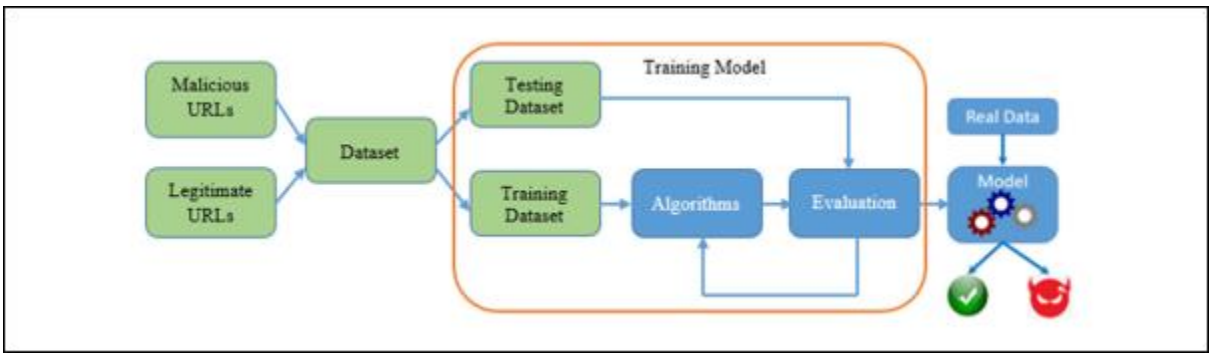


Fig 3.1 existing system architecture

3.2.2 LIMITATIONS OF THE EXISTING SYSTEM

- Traditional systems can't detect new (zero-day) phishing attacks because they rely on outdated blacklists and fixed patterns.
- They lack deep analysis of web content and server-side security, making them ineffective against modern phishing tactics hidden in HTML and links.

3.3 PROPOSED SYSTEM

The WebSec Analyzer aims to overcome the limitations of traditional security systems by introducing a comprehensive and intelligent solution that integrates real-time AI-driven phishing detection with deep vulnerability analysis. Unlike conventional methods that rely heavily on static blacklists and pattern matching, the proposed system uses a trained machine learning model capable of identifying zero-day phishing threats by analyzing the underlying structure and features of URLs and web content. Additionally, it performs thorough scanning of websites to uncover security loopholes, such as open ports, outdated software, and weak configurations, using advanced scanning tools and techniques. The system combines front-end user interaction with a powerful backend that processes data dynamically and presents actionable insights in an intuitive interface. By integrating phishing detection and

vulnerability assessment into a single framework, the WebSec Analyzer provides a robust defense mechanism against a wide range of cybersecurity threats.

3.3.1 PROPOSED SYSTEM ARCHITECTURE

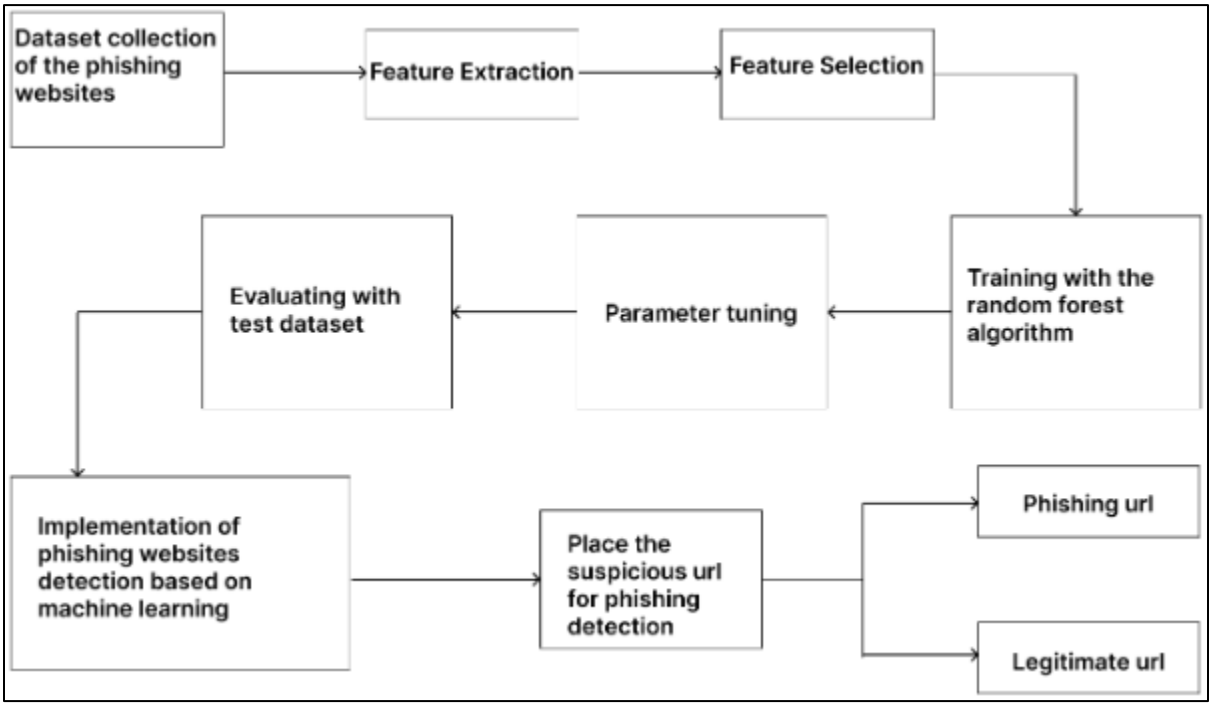


Fig 3.2 proposed system architecture

Phishing Detector Module:

- Uses ML algorithms (SVM, Random Forest, Deep Learning).
- Extracts features from URL, HTML content, redirections, domain age, etc.
- Classifies sites as phishing or legitimate based on trained models.

Vulnerability Analyzer Module:

- Inspects HTTP response headers (CSP, X-Frame-Options, etc.).
- Performs WHOIS and SSL certificate checks.
- Outputs a risk report.

USE CASE DIAGRAM

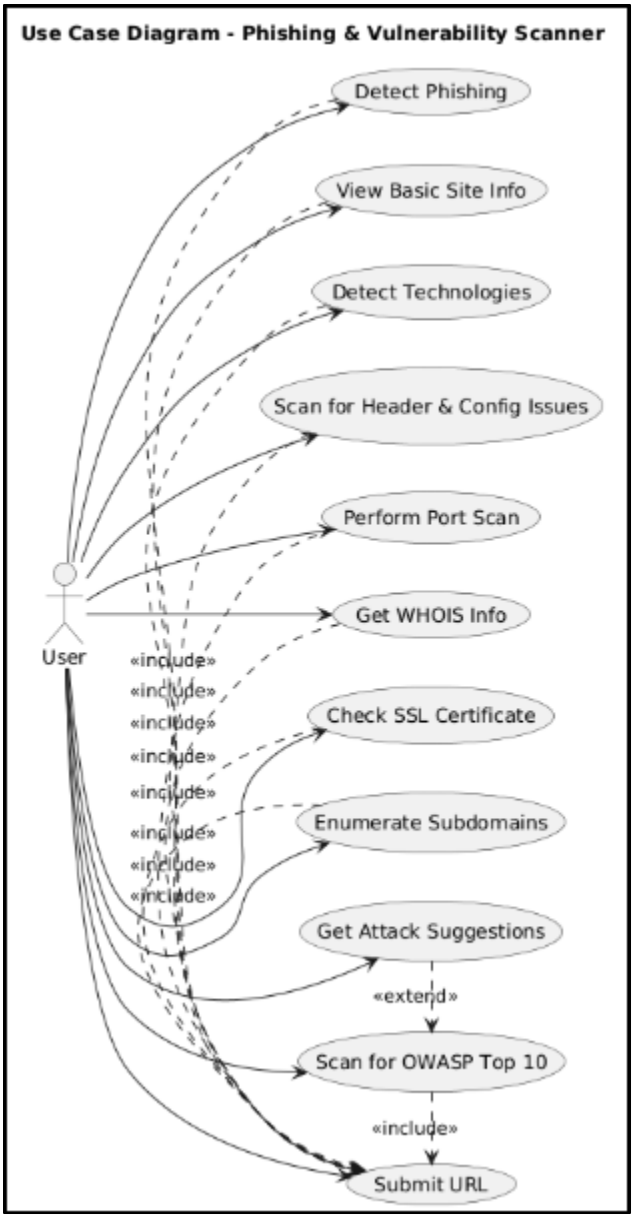


Fig 3.3 Use Case Diagram

CLASS DIAGRAM FOR PROPOSED SYSTEM

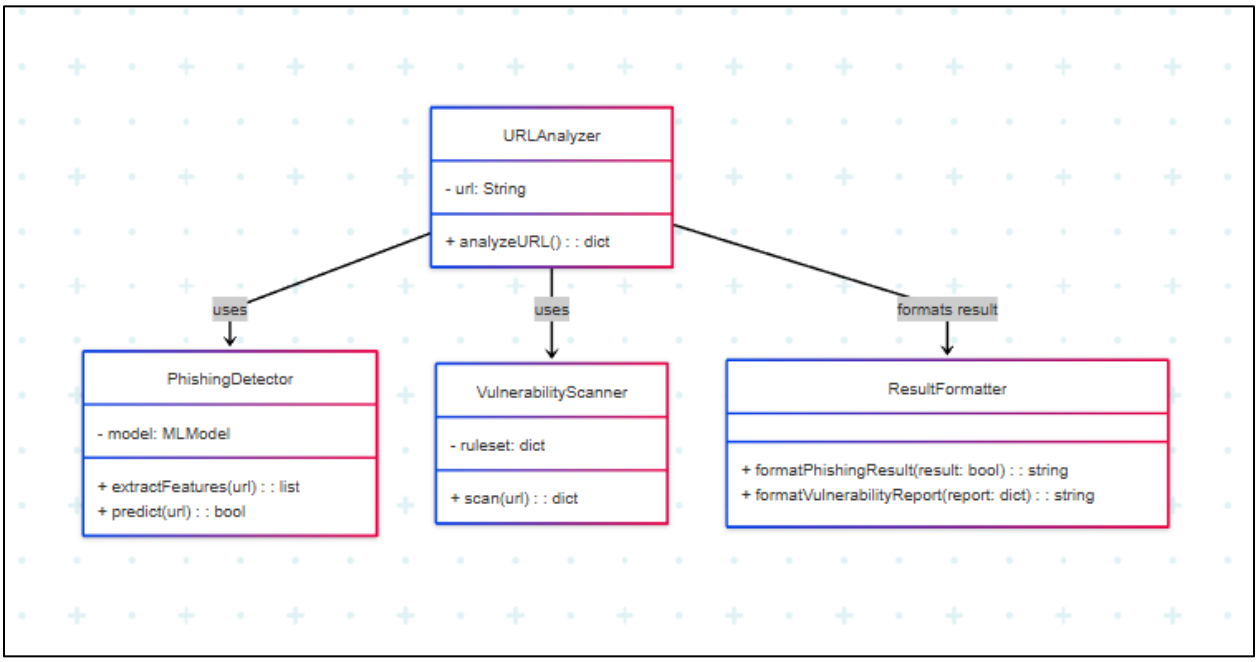


Fig 3.4 Class Diagram

SEQUENCE DIAGRAM FOR PROPOSED SYSTEM

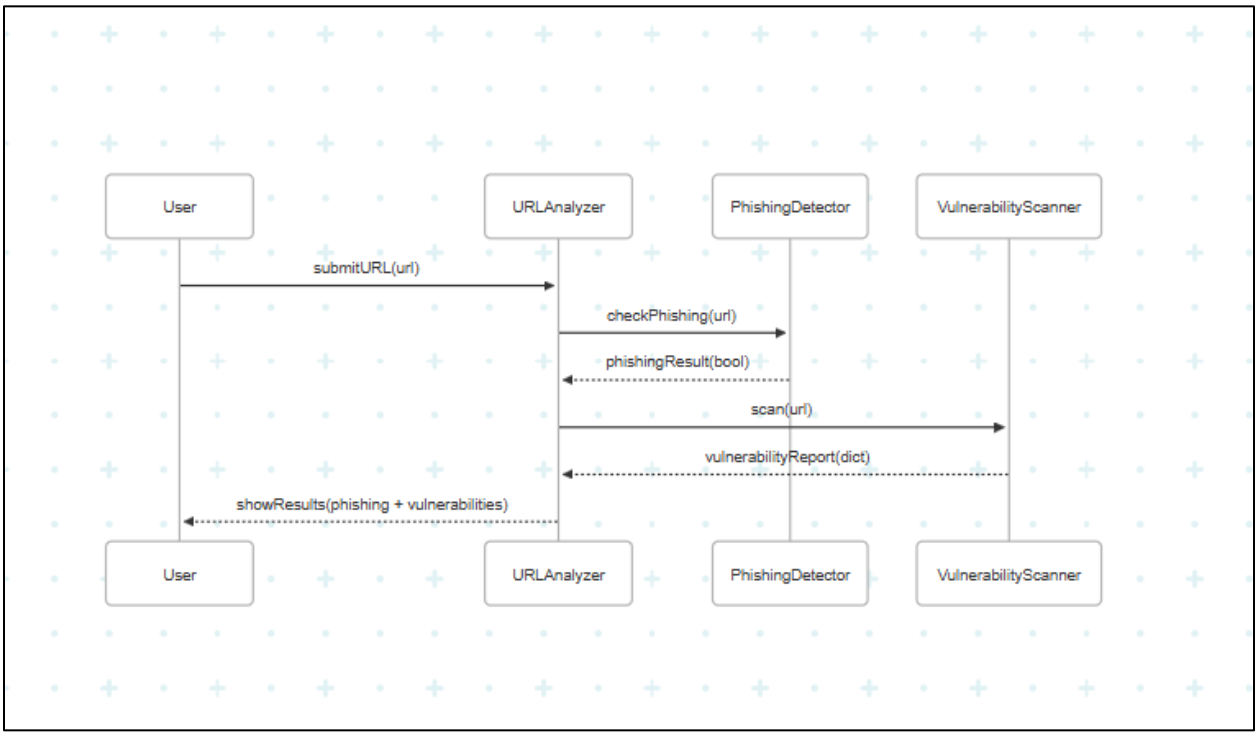


Fig 3.5 Sequence Diagram

3.3.2 ADVANTAGES OF PROPOSED SYSTEM

- Accurate AI-based phishing detection that works well even with new threats.
- Real-time scanning with easy-to-understand reports.
- Detailed domain info using SSL, WHOIS, and other checks.
- Simple, user-friendly dashboard useful for both regular users and security experts.

3.4 SUMMARY

The WebSec Analyzer system is a hybrid platform designed to bridge the gap between phishing detection and vulnerability assessment. It improves security awareness and strengthens risk management by delivering timely insights and automated analysis. Leveraging modern AI techniques and cybersecurity methodologies, the system ensures a more proactive and intelligent approach to identifying and mitigating threats.

CHAPTER 4

SYSTEM DESIGN AND IMPLEMENTATION

4.1 GENERAL

The WebSec Analyzer project is designed to provide a comprehensive cybersecurity solution by combining vulnerability analysis and AI-based phishing detection into a unified web platform. It aims to help users evaluate the security posture of websites and detect phishing threats using machine learning and heuristic analysis. The system architecture involves both frontend and backend components, integrating real-time security scanning with intelligent threat classification. Built with modern web technologies, this system ensures accessibility, user interaction, and efficient data processing for security insights.

4.2 LIST OF MODULES

- User Interface & API Module
- AI-Based Phishing Detection Module
- Basic Website Information Extraction
- Technology Stack Detection
- Security Header & Configuration Scanner
- Open Port Scanning
- Domain WHOIS & SSL Certificate Analysis
- OWASP Top 10 Vulnerability Scanner with Attack Suggestions

4.3 MODULE DESCRIPTION

The modules are performed using different techniques and they are described below with the help of illustrations.

4.3.1 User Interface & API Module

This module serves as the bridge between the frontend and backend. The API is exposed via the /analyze endpoint, which receives a URL as input in JSON format from the frontend. It orchestrates all the analysis tasks—phishing detection, vulnerability scanning, and metadata extraction—by calling various backend functions. The results are then compiled into a structured JSON response and sent back to the frontend for display.

4.3.2 AI-Based Phishing Detection Module

This module uses a trained machine learning model (phishing_detector.pkl) to detect phishing websites. It performs feature extraction from the given URL, which includes:

- Presence of IP address in the URL
- Use of '@' symbol
- Length of the URL
- Number of dots in the URL
- Domain age (calculated using WHOIS data)

These features are passed to the model for classification. The output indicates whether the website is Phishing or Legitimate.

4.3.3 Basic Website Information Extraction

This component sends an HTTP request to the target website and parses its content using BeautifulSoup. It extracts:

- Page title
- Server type from response headers
- HTTP status code (e.g., 200 OK, 404 Not Found)

This provides a basic understanding of the website's identity and responsiveness.

4.3.4 Technology Stack Detection

This module identifies the technologies used by the website:

- Web frameworks (e.g., Flask, Django, Laravel, Rails)
- CMS platforms (e.g., WordPress, Joomla, Drupal)
- Server software (e.g., Apache, Nginx)
- X-Powered-By header indicating backend tech

The detection is done by scanning response headers and body content for keywords, cookies, and resource paths.

4.3.5 Security Header And Configuration Scanner

This module checks the presence of essential HTTP security headers that help prevent common web attacks:

- Content-Security-Policy
- X-Frame-Options
- X-XSS-Protection
- Strict-Transport-Security
- Referrer-Policy
- Permissions-Policy

It also detects if directory listing is enabled by analyzing the page content for "Index of /", which may reveal sensitive files to attackers.

4.3.6 Open Port Scanning

The system scans a predefined set of common TCP ports (e.g., 21, 22, 80, 443, 3306) to identify services running on the domain. Using Python's socket library and multithreading, it quickly identifies open ports, which could indicate potential entry points for attackers.

4.3.7 Domain WHOIS & SSL Certificate Analysis

This module retrieves domain registration data using the WHOIS protocol:

- Registrar name
- Creation and expiration dates
- Name servers

It also checks the website's SSL certificate using Python's ssl and socket modules:

- Expiry date of the certificate
- Days remaining before expiration

Insecure or expired SSL certificates are flagged for user awareness.

4.3.8 OWASP Top 10 Vulnerability Scanner with Attack Suggestions

This component passively scans the website content and headers to detect patterns linked to the OWASP Top 10 vulnerabilities:

- A01: Broken Access Control
- A02: Cryptographic Failures
- A03: Cross-Site Scripting (XSS)
- A04: Information Exposure
- A05: Security Misconfiguration
- A06: Vulnerable Components
- A07: Identification and Authentication Failures
- A08: Insecure Software Loading
- A10: Server-Side Request Forgery (SSRF)

Based on identified vulnerabilities, the module suggests potential attack vectors (e.g., XSS payloads, MITM attacks, SSRF testing). This helps security researchers understand how attackers might exploit the flaws.

4.4 SUMMARY

In summary, WebSec Analyzer integrates advanced phishing detection with deep website vulnerability assessment. It is structured with modular components ensuring scalability and ease of maintenance. The system enhances web security awareness by offering tools to proactively detect and mitigate risks through real-time analysis and machine learning.

CHAPTER 5

SYSTEM REQUIREMENT

5.1 GENERAL

This chapter explains about the software and hardware requirements and their specifications. The requirements listed here are used to satisfy system design and other implementation design.

5.2 SYSTEM REQUIREMENTS

5.2.1 HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or equivalent (Dual Core)
- RAM: 4 GB (8 GB Recommended)
- Storage: 500 MB free space
- Display: 13" monitor or higher (minimum 720p)
- Network : Internet connectivity required

5.2.2 SOFTWARE REQUIREMENTS

Software / Tools	Purpose
Windows / Linux / macOS	Operating System
Python 3.8+	Backend programming language
Flask	To build the backend API
joblib	To load the trained ML model
BeautifulSoup (bs4)	For analyzing website HTML content
requests	To send and receive HTTP requests
socket	For network and port scanning
whois	To get domain registration details
ssl	To check SSL certificates
NumPy	To process feature data
React.js	To build the frontend user interface
Node.js	To run the frontend application
Visual Studio Code (IDE)	Code editor used for development

5.3 TECHNICAL SPECIFICATION

5.3.1 Python (Core Language)

Python serves as the core programming language for the backend of the system. It is used for implementing machine learning-based phishing detection, web scraping, HTTP analysis, vulnerability scanning, and domain information retrieval.

Python was selected due to its:

- Wide range of libraries for cybersecurity, machine learning, and network communication
- Strong community support
- Readability and ease of use

5.4 SUMMARY

The WebSec Analyzer system is developed using lightweight, open-source technologies suitable for cross-platform deployment. It requires moderate hardware resources and standard software tools such as Python and React.js. The backend leverages Python libraries like Flask, joblib, and BeautifulSoup for web analysis and machine learning operations, while the frontend is built using React.js for a responsive and interactive user interface. The system ensures efficient performance on machines with basic specifications and is compatible with Windows, Linux, or macOS platforms, making it accessible and easy to deploy for developers and cybersecurity professionals.

CHAPTER 6

CONCLUSION

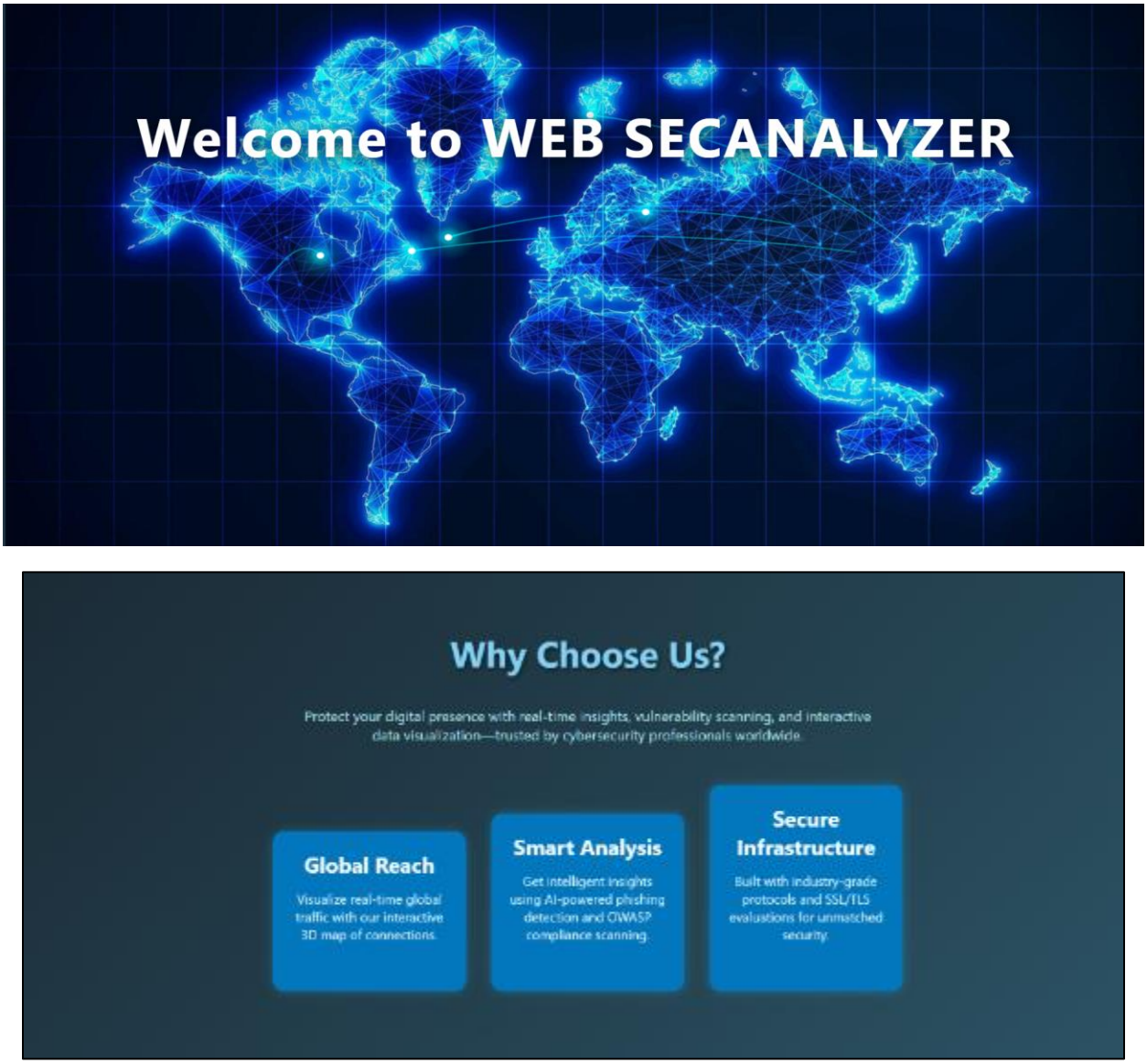
The WebSec Analyzer project successfully demonstrates an integrated approach to cybersecurity by combining AI-based phishing detection with comprehensive vulnerability analysis. Through the use of machine learning algorithms, URL and HTML feature extraction, and heuristic techniques, the system efficiently identifies phishing websites. Additionally, it evaluates various security aspects of a target website, including HTTP headers, SSL certificates, open ports, WHOIS data, and compliance with OWASP Top 10 vulnerabilities.The modular architecture, built using Python for the backend and React.js for the frontend, ensures flexibility, scalability, and user-friendliness. The system can analyze both the visible content and underlying security configurations of a website, making it a practical and educational tool for students, developers, and cybersecurity enthusiasts. This project showcases how artificial intelligence and traditional cybersecurity practices can be effectively combined to build automated, intelligent, and reliable security tools.

FUTURE ENHANCEMENT

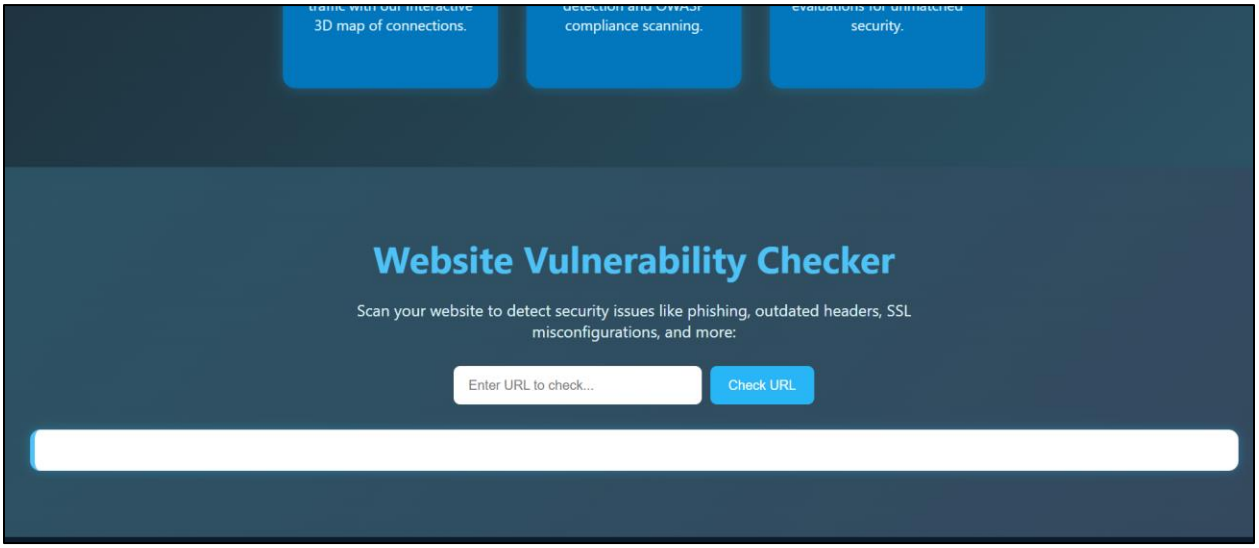
In the future, WebSec Analyzer can be improved by adding real-time URL monitoring to detect phishing sites instantly as users visit them. More advanced machine learning models like deep learning can be used to increase detection accuracy. Features like downloadable scan reports (PDF/CSV), browser extensions for instant alerts, and storing scan history for future reference can also be added. Integration with popular security tools like OWASP ZAP will help in deeper vulnerability scanning. Adding support for multiple languages will make the tool more user-friendly for a wider audience.

APPENDIX 1

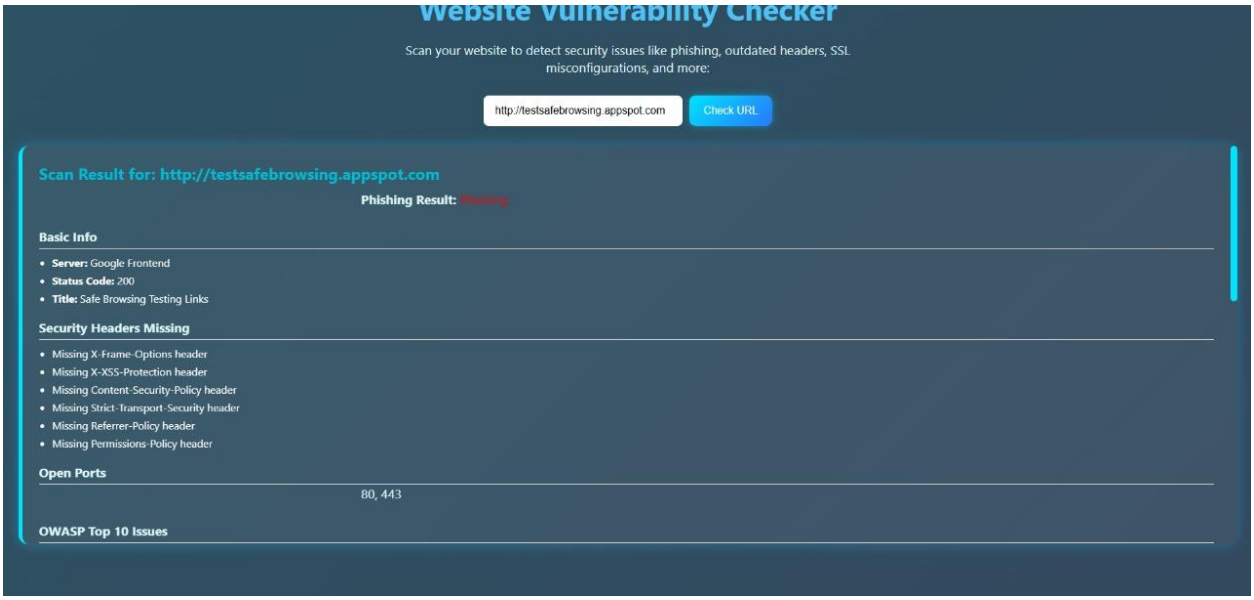
SCREENSHOTS

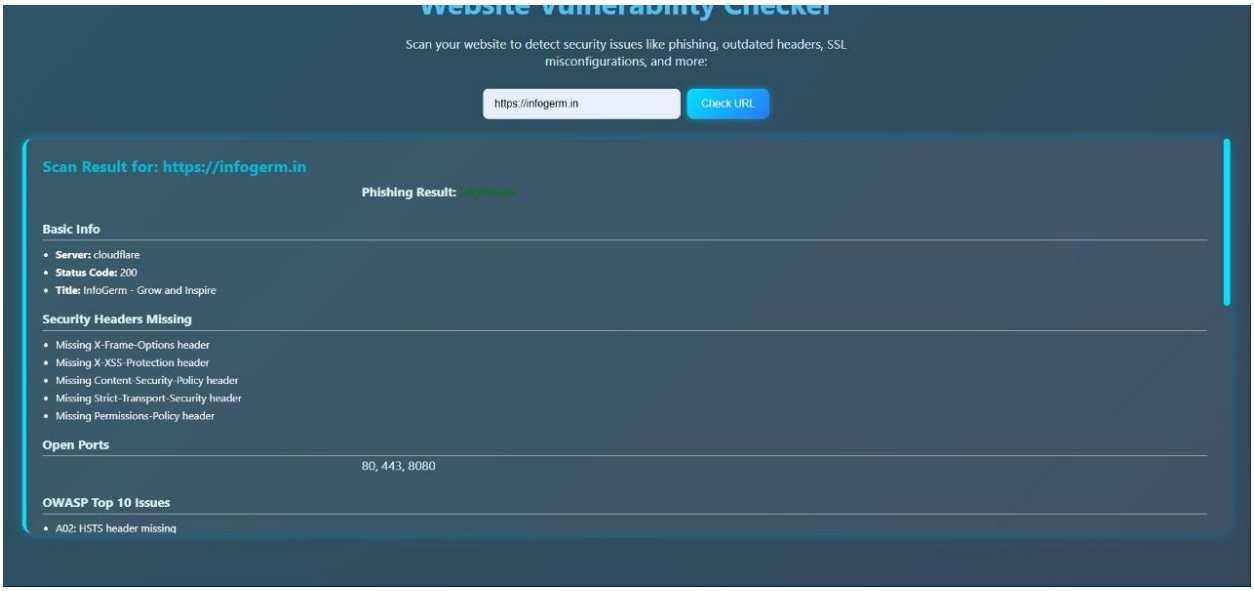


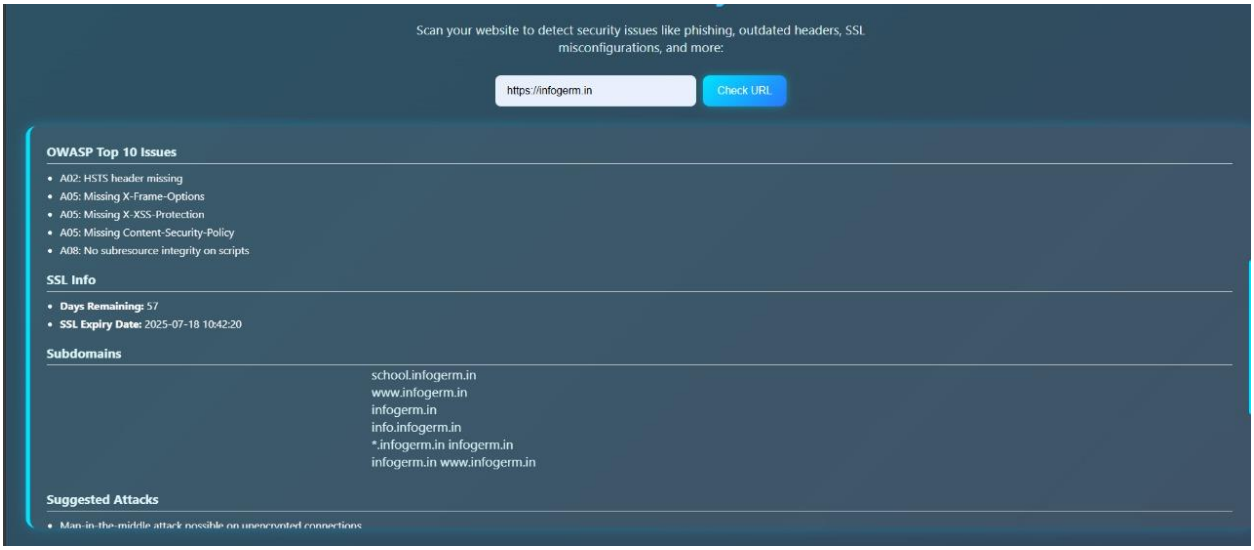
A1.1 Home Page

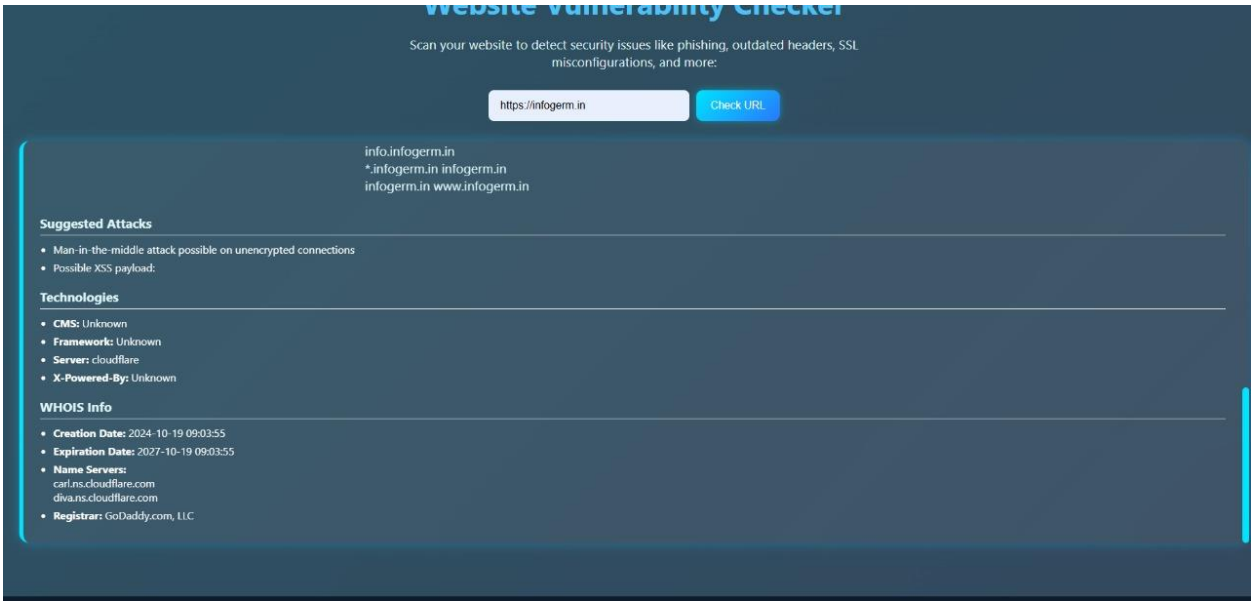


A1.2 URL Box to Check




Website Vulnerability Checker
Scan your website to detect security issues like phishing, outdated headers, SSL misconfigurations, and more:
https://infogerm.in
Check URL
Scan Result for: https://infogerm.in
Phishing Result:
Basic Info
Server: cloudflare
Status Code: 200
Title: InfoGerm - Grow and Inspire
Security Headers Missing
Missing X-Frame-Options header
Missing X-XSS-Protection header
Missing Content-Security-Policy header
Missing Strict-Transport-Security header
Missing Permissions-Policy header
Open Ports
80, 443, 8080
OWASP Top 10 Issues
A02: HSTS header missing


Scan your website to detect security issues like phishing, outdated headers, SSL misconfigurations, and more:
https://infogerm.in
Check URL
OWASP Top 10 Issues
A02: HSTS header missing
A05: Missing X-Frame-Options
A05: Missing X-XSS-Protection
A05: Missing Content-Security-Policy
A08: No subresource integrity on scripts
SSL Info
Days Remaining: 57
SSL Expiry Date: 2025-07-18 10:42:20
Subdomains
school.infogerm.in
www.infogerm.in
infogerm.in
info.infogerm.in
*.infogerm.in infogerm.in
infogerm.in www.infogerm.in
Suggested Attacks



A1.3 Vulnerability Analyzer and Phishing Detection

APPENDIX 2

SAMPLE CODING

Html.css:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>WebSec Analyzer - Global Security Map</title>
  <link href="https://unpkg.com/aos@2.3.4/dist/aos.css" rel="stylesheet" />
  <style>
    body {
      font-family: 'Segoe UI', sans-serif;
      background: linear-gradient(135deg, #0f2027, #203a43, #2c5364);
      color: #fff;
      margin: 0;
    }
    header {
      height: 100vh;
      display: flex;
      justify-content: center;
```

```
  align-items: center;
  position: relative;
}
header h1 {
  font-size: 4rem;
  position: absolute;
  top: 20%;
  text-align: center;
}
canvas.heroCanvas {
  position: absolute;
  width: 100%;
  height: 100%;
}
section {
  padding: 60px 20px;
  text-align: center;
}
.vulnerability-checker {
  display: flex;
  justify-content: center;
```

```
    gap: 10px;
    flex-wrap: wrap;
  }
  .results {
    background: #fff;
    color: #000;
    margin-top: 20px;
    padding: 20px;
    border-radius: 10px;
  }
  footer {
    background: #0d1b2a;
    color: #aaa;
    text-align: center;
    padding: 20px;
  }
</style></head><body>  <header>
  <canvas id="mapCanvas" class="heroCanvas"></canvas>
  <h1 data-aos="zoom-in">Welcome to WebSec Analyzer</h1>
</header>  <section id="features" data-aos="fade-up">
  <h2>Why Choose Us?</h2>
```

We provide global visualization, intelligent threat analysis, and secure infrastructure insights.

</section> <section id="scanner" data-aos="fade-up">

<h2>Website Vulnerability Checker</h2>

<input type="text" id="urlInput" placeholder="Enter URL to check..." />

<button onclick="checkVulnerability()">Check URL</button>

</section> <footer>

© 2025 WebSec Analyzer. All rights reserved. | Privacy Policy

</footer>

<script src="https://unpkg.com/aos@2.3.4/dist/aos.js"></script>

<script>AOS.init({ duration: 1000, once: true });</script>

<script>

const canvas = document.getElementById("mapCanvas");

const ctx = canvas.getContext("2d");

canvas.width = 1250;

canvas.height = 700;const mapImage = new Image();

mapImage.src = "https://your-cdn.com/world-map.jpg"; // Sample background

const nodes = {

"New York": { x: 500, y: 500 },

```
    "London": { x: 820, y: 450 }
  };
  mapImage.onload = () => {
    ctx.drawImage(mapImage, 0, 0, canvas.width, canvas.height);
    ctx.strokeStyle = "#29b6f6";
    ctx.beginPath();
    ctx.moveTo(nodes["New York"].x, nodes["New York"].y);
    ctx.lineTo(nodes["London"].x, nodes["London"].y);
    ctx.stroke();
  };
 </script> <script>
  function checkVulnerability() {
    const url = document.getElementById("urlInput").value;
    const resultsDiv = document.getElementById("results");
    resultsDiv.innerHTML = `<h3>Scan Results for:</h3><p>${url}</p><ul><li>SSL: Secure</li><li>Phishing: Not Detected</li></ul>`;
  }
 </script></body></html>
```

App.py:

from flask import Flask, request, jsonify from flask_cors import CORS import joblib import numpy as np import re import urllib import requests import whois import datetime import socket

app = Flask(name) CORS(app)

model = joblib.load("phishing_detector.pkl")

def extract_domain(url): try: return urllib.parse.urlparse(url).netloc except: return ''

def has_ip(url): return 1 if re.search(r'\d+.\d+.\d+.\d+', url) else 0

def has_at_symbol(url): return 1 if '@' in url else 0

def url_length(url): return len(url)

def count_dots(url): return url.count('.')

def domain_age(domain_name): try: info = whois.whois(domain_name) creation_date = info.creation_date if isinstance(creation_date, list): creation_date = creation_date[0] if creation_date: return (datetime.datetime.now() - creation_date).days else: return -1 except: return -1

def get_features(url): domain = extract_domain(url) return [has_ip(url), has_at_symbol(url), url_length(url), count_dots(url), domain_age(domain)]@app.route("/analyze", methods=["POST"]) def analyze(): data = request.get_json() url = data.get("url") if not url: return jsonify({"error": "URL is required"}), 400

domain = extract_domain(url) features = np.array(get_features(url)).reshape(1, -1)

```
prediction = model.predict(features)[0]
phishing_result = 'Phishing' if prediction == 1 else 'Legitimate'

return jsonify({
    "url": url,
    "domain": domain,
    "phishing_result": phishing_result
})

if name == "main": app.run(debug=True, host="0.0.0.0", port=5000)
```

1.HomePage (Frontend)

```
<header>
  <canvas id="mapCanvas" class="heroCanvas"></canvas>
  <h1 data-aos="zoom-in">Welcome to WEB SEC ANALYZER</h1>
</header>

<section class="features" id="features" data-aos="fade-up">
  <h2>Why Choose Us?</h2>
  <p>Protect your digital presence with real-time insights, vulnerability scanning, and interactive data visualization—trusted by cyber
  <div class="feature-card" data-aos="fade-up" data-aos-delay="100">
    <h3>Global Reach</h3>
    <p>Visualize real-time global traffic with our interactive 3D map of connections.</p>
  </div>
  <div class="feature-card" data-aos="fade-up" data-aos-delay="200">
    <h3>Smart Analysis</h3>
    <p>Get intelligent insights using AI-powered phishing detection and OWASP compliance scanning.</p>
  </div>
  <div class="feature-card" data-aos="fade-up" data-aos-delay="300">
    <h3>Secure Infrastructure</h3>
    <p>Built with industry-grade protocols and SSL/TLS evaluations for unmatched security.</p>
  </div>
</section>
```

A2.1 Hero Section

```
<section id="scanner" data-aos="fade-up">
  <h2>Website Vulnerability Checker</h2>
  <p>Scan your website to detect security issues like phishing, outdated headers, SSL misconfigurations, and more:</p>
  <div class="vulnerability-checker">
    <input type="text" id="urlInput" placeholder="Enter URL to check..." />
    <button onclick="checkVulnerability()">Check URL</button>
  </div>
  <div class="results" id="results"></div>
</section>

<footer>
  <p>&copy; 2025 Your Company Name. All Rights Reserved. |
    <a href="#">Privacy Policy</a> |
    <a href="#">Terms of Service</a>
  </p>
</footer>

<script src="https://unpkg.com/aos@2.3.4/dist/aos.js"></script>
<script> AOS.init({
    duration: 1000,
    once: true
  });</script>
```

A2.2 URL Box

2.Vulnerability Analyzer and Phishing Detection (Backend)

app.py | phishing_detector.pkl | index....html | requirements.txt

app.py > analyze

```python
    return open_ports if open_ports else ['No open common ports found']

def get_domain_info(domain):
    try:
        info = whois.whois(domain)
        return {
            'Registrar': info.registrar,
            'Creation Date': str(info.creation_date),
            'Expiration Date': str(info.expiration_date),
            'Name Servers': info.name_servers
        }
    except Exception as e:
        return {'Error': str(e)}

def check_ssl(domain):
    try:
        ctx = ssl.create_default_context()
        with socket.create_connection((domain, 443)) as sock:
            with ctx.wrap_socket(sock, server_hostname=domain) as ssock:
                cert = ssock.getpeercert()
                expiry = datetime.datetime.strptime(cert['notAfter'], '%b %d %H:%M:%S %Y GMT')
                days = (expiry - datetime.datetime.utcnow()).days
                return {'SSL Expiry Date': str(expiry), 'Days Remaining': days}
    except Exception as e:
        return {'Error': str(e)}

def find_subdomains(domain):
    url = f"https://crt.sh/?q=%.{domain}&output=json"
    try:
        r = requests.get(url, timeout=10)
        data = r.json()
        subdomains = set(entry['name_value'] for entry in data)
        return list(subdomains)
    except Exception as e:
        return [f"Error: {e}"]

def check_owasp_top10(url):
```

A2.3 Vulnerability Analyzer

app.py | phishing_detector.pkl | index....html | requirements.txt

app.py > analyze

```
228 def suggest_attacks(findings):
239         elif "admin page" in finding:
240             suggestions.append("Try default credentials or directory traversal")
241     return suggestions if suggestions else ["No attack suggestions"]
242
243 # --- Unified Analysis Route ---
244 @app.route("/analyze", methods=["POST"])
245 def analyze():
246     data = request.get_json()
247     url = data.get("url")
248     domain = extract_domain(url)
249
250     phishing_features = np.array(get_features(url)).reshape(1, -1)
251     prediction = model.predict(phishing_features)[0]
252     phishing_result = 'Phishing' if prediction == 1 else 'Legitimate'
253
254     results = {
255         'phishing_result': phishing_result,
256         'basic_info': get_website_info(url),
257         'technologies': detect_technologies(url),
258         'headers_and_config': scan_headers_and_config(url),
259         'open_ports': scan_ports(domain),
260         'whois': get_domain_info(domain),
261         'ssl_info': check_ssl(domain),
262         'subdomains': find_subdomains(domain),
263         'owasp_top10': check_owasp_top10(url)
264     }
265
266     results['suggested_attacks'] = suggest_attacks(results['owasp_top10'])
267
268     return jsonify(results)
269
270 if __name__ == "__main__":
271     app.run(debug=True, host='0.0.0.0', port=5000)
272
```

A2.4 Phishing Detection

REFERENCES

1. Anti-Phishing Working Group. (Sep. 2022). Phishing Attacks Trends Report-Q2 2022. Accessed: Oct. 15, 2022. [Online]. Available: https:// apwg.org/trendsreports/

2. Cloudflare's 2023 Phishing Threats Report. Accessed: Oct. 1, 2023. [Online]. Available: https://www.cloudflare.com/lp/2023-phishing-report/

3. M. Volkamer, K. Renaud, B. Reinheimer, and A. Kunz, ''User expe riences of TORPEDO: Tooltip-powered phishing email detection,'' Comput. Secur., vol. 71, pp. 100–113, Nov. 2017, doi: 10.1016/j.cose. 2017.02.004.

4. N. Q. Do, A. Selamat, O. Krejcar, E. Herrera-Viedma, and H. Fujita, ''Deep learning for phishing detection: Taxonomy, current challenges and future directions,'' IEEE Access, vol. 10, pp. 36429–36463, 2022, doi: 10.1109/ACCESS.2022.3151903.

5. T. Mahara, V. L. H. Josephine, R. Srinivasan, P. Prakash, A. D. Algarni, and O. P. Verma, ''Deep vs. shallow: A comparative study of machine learning and deep learning approaches for fake health news detection,'' IEEE Access, vol. 11, pp. 79330–79340, 2023, doi: 10.1109/ACCESS.2023.3298441.

6. Google Safe Browsing. Accessed: Oct. 1, 2023. [Online]. Available: https://safebrowsing.google.com/

7. (2019). Office 365 Advanced Threat Protection Safe Links. Accessed: Jul. 10, 2023. [Online]. Available: https://docs.microsoft.com/en us/office365/securitycompliance/atp-safe-links